

UNITAT 3 · SETMANES 3–4

Fitxers i sistemes de fitxers

📅 Setmanes 3–4 · dates per confirmar <> Laboratori TD2 (resolt + repte IA) ~ 4 animacions

MANUAL M3 · TEORIA DE LA UNITAT

Fitxers i sistemes de fitxers

Lectura bàsica de la unitat 3 ($\approx 1,5$ h). Cada secció acaba amb un enllaç **Practica-ho** →. Les sortides són reals: s'han obtingut amb els programes del TD2 a la màquina del curs (Ubuntu 26.04, Linux 7.0, ext4 sobre NVMe).

CONTINGUT

- | | |
|--|---|
| §1 Dos nivells: el disc i el nucli | §5 Fitxers especials: FIFO, dispositius, <code>/proc</code> |
| §2 L'inode: metadades i blocs | §6 Recórrer directoris |
| §3 Els noms: directoris, enllaços i camins | §7 Més enllà: registre de diari, còpia en escriptura i <code>fsync</code> |
| §4 Fitxers oberts: les tres taules | |

§1 Dos nivells: el disc i el nucli

Un fitxer existeix en dos llocs. **Al disc**, el sistema de fitxers organitza una enorme matriu de blocs (4 KiB a la màquina del curs): un *superbloc* amb la descripció general, mapes de bits que diuen quins blocs i inodes són lliures, les *taules d'inodes* i els *blocs de dades*. ext4 repeteix aquesta estructura en *grups de blocs* al llarg de la partició, amb còpies del superbloc. El nombre d'inodes es fixa en formatar:

Sortida real

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /
block size: 4096 bytes
```

Al nucli, mentre un procés fa servir el fitxer, hi ha tres taules: la taula de descriptors de cada procés, la taula de fitxers oberts del sistema i la taula d'inodes a la memòria (§4). Per sobre de tot, la *memòria cau de pàgines* guarda a la RAM els blocs llegits recentment: per això la segona lectura d'un fitxer gairebé mai no toca el disc.

Mira-ho a la presentació P3 →

§2 L'inode: metadades i blocs

Unix separa el **contingut** d'un fitxer de les seves **metadades**. Les metadades viuen en un **inode**, identificat per un número: tipus i permisos (`st_mode`), nombre d'enllaços durs (`st_nlink`), propietari i grup, mida, dispositiu, marques de temps i la llista de blocs. L'inode **no** conté el nom. `stat`, `fstat` (sobre un descriptor) i `lstat` (sense seguir enllaços) el llegeixen:

Sortida real

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096   regular file
Device: 259,5      Inode: 12190686   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: ( 1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

Les quatre marques de temps: **Access** (última lectura; Linux l'actualitza de manera mandrosa amb `relatime`), **Modify** (últim canvi del contingut), **Change** (últim canvi de l'inode: permisos, enllaços, propietari... o contingut) i **Birth** (creació, disponible a ext4 i que es llegeix amb `statx`). És un error freqüent —les diapositives originals el feien— pensar que `ctime` és la creació.

El tipus es comprova amb les macros `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISCHR`, `S_ISBLK`, `S_ISFIFO` i `S_ISSOCK` sobre `st_mode`. Els errors típics de `stat` són `ENOENT` (no existeix), `EACCES` (falta el permís de cerca `x` en un directori del camí), `ENOTDIR` i `ELOOP`.

On són els blocs? ext2 i ext3 desaven 15 punters a l'inode: 12 directes, un indirecte simple (un bloc ple de punters), un de doble i un de triple. Amb blocs de 4 KiB i punters de 4 bytes, en caben 1024 per bloc i l'esquema arriba a uns 4 TiB, però una posició llunyana obliga a llegir fins a 3 blocs d'índexs. ext4 fa servir **extents**: trams contigus descrits per (inici lògic, longitud ≤ 32.768 blocs, inici físic). En caben quatre a l'inode; si en calen més, es construeix un arbre. El fitxer de 10 MB que escriu el TD2 només n'ocupa dos:

Sortida real

```
$ head -c 10000000 /dev/urandom > big.bin; sync big.bin; filefrag -v big.bin
Filesystem type is: ef53
File size of big.bin is 10000000 (2442 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..      2047:   15654912..   15656959:   2048:
  1:      2048..      2441:   15673843..   15674236:    394:   15656960: last,eof
big.bin: 2 extents found
```

§3 Els noms: directoris, enllaços i camins

Un **director** és un fitxer les dades del qual són una llista de parelles *nom* → *número d'inode*, més `.` (ell mateix) i `..` (el seu pare). Per obrir `/home/student/notas.txt` el nucli comença a l'inode arrel (el número 2 a ext4), llegeix el directori, busca `home`, continua amb el seu inode, i així component a component (*path_resolution(7)*). Cada component exigeix permís de cerca (`x`) sobre el directori. Linux guarda les resolucions recents a la *memòria cau de dentries*.

Un **enllaç dur** (`link`, `ln`) és una altra entrada que apunta al **mateix** inode: tots dos noms són igual d'«originals» i `st_nlink` els compta. No es permeten a directoris (trencarien l'arbre) ni entre sistemes de fitxers (els números d'inode són locals). Un **enllaç simbòlic** (`symlink`, `ln -s`) és un inode nou el contingut del qual és un *camí*; la resolució el segueix (com a màxim 40 vegades: després, `ELOOP`) i pot quedar penjat si desapareix la destinació.

Sortida real

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

`rm` crida `unlink`, que elimina una **entrada** i resta 1 a `st_nlink`. El nucli només allibera l'inode i els seus blocs quan `st_nlink = 0` i cap procés no el té obert. Mentrestant el fitxer encara es pot llegir pel seu descriptor:

Sortida real

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l
atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

⚠ Comprovar i després fer servir: la cursa TOCTOU

`if (access(path, W_OK) == 0) fd = open(path, O_WRONLY);` té una finestra: entre les dues crides un altre procés pot substituir `path` per un enllaç a un altre fitxer. Obre directament i gestiona l'error (`EACCES`, `ENOENT`), o fes servir `O_NOFOLLOW` o `openat2()` amb `RESOLVE_NO_SYMLINKS`. A més, `access` comprova amb l'UID *real*, no amb l'efectiu.

Practica-ho → A3.1 Inodes i directoris

§4 Fitxers oberts: les tres taules

`open(path, flags, mode)` resol el camí, comprova els permisos i retorna el **descriptor lliure més baix**. Els indicadors: accés (`O_RDONLY`, `O_WRONLY`, `O_RDWR`), creació (`O_CREAT`, `O_EXCL`, `O_TRUNC`); els permisos demanats passen per l' `umask` i estat (`O_APPEND`, `O_NONBLOCK`, `O_CLOEXEC`). glibc implementa `open` amb `openat(AT_FDCWD, ...)`, que és el que mostra `strace`.

Darrere de cada descriptor hi ha tres nivells:

1. **Taula de descriptors**, una per procés: el número (0, 1, 2, 3...) n'és un índex.
2. **Taula de fitxers oberts**, una per a tot el sistema: cada `open` crea una entrada amb el mode, la **posició** actual i un comptador de referències.
3. **Taula d'inodes a la memòria**: una còpia de l'inode per fitxer, compartida per totes les obertures.

Per això importa qui comparteix cada entrada. Dues crides `open` sobre el mateix fitxer creen dues entrades amb posicions independents; `dup`, `dup2` i `fork` copien el *punter* i comparteixen la posició. Amb `donnees.txt = abcdefgh`:

Sortida real

```
$ ./alias_open
abcdabcd
```

Sortida real

```
$ ./alias_dup
abcdefgh
```

Conseqüències pràctiques: `./prog > out 2> out` obre `out` dues vegades i la sortida d'error sobreescriu la sortida estàndard; `./prog > out 2>&1` fa servir `dup2` i no perd res. Amb `O_APPEND`, cada `write` es mou al final de manera atòmica: així escriuen els servidors els seus registres. `close` resta una referència; l'entrada s'allibera quan arriba a 0.

Practica-ho → A3.2 Les tres taules · TD2 · 3

§5 Fitxers especials: FIFO, dispositius, `/proc`

«Tot és un fitxer» vol dir que moltes coses que no són dades en un disc fan servir la mateixa interfície `open` / `read` / `write` / `close`. Una **FIFO** (canonada amb nom, `mkfifo`) és un inode sense blocs: el que un procés escriu, un altre ho llegeix a través d'una memòria intermèdia de 64 KiB del nucli. Les seves regles, que fa servir el xat del TD2:

- `open(O_RDONLY)` espera fins que algú obre per escriure, i al revés (amb `O_NONBLOCK`, un escriptor sense lector rep `ENXIO`).
- Una escriptura de fins a `PIPE_BUF = 4096` bytes és **atòmica**: no es barreja amb la d'un altre escriptor. Una de més gran es pot barrejar.

- `read` retorna 0 (final de fitxer) quan tanca l'últim escriptor.
- Escriure sense cap lector produeix el senyal `SIGPIPE` (o l'error `EPIPE` si s'ignora).

Altres fitxers especials: els **dispositius** de caràcters i de blocs de `/dev` (creats amb `mknod`), els **sòcols** del domini UNIX (unitat 8 i TD8) i els sistemes de fitxers virtuals `/proc` i `/sys`, els «fitxers» dels quals genera el nucli quan es llegeixen.

Sortida real

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' | ./fifo_client
chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Practica-ho → A3.3 FIFO: el xat · TD2 · 2

§6 Recórrer directoris

`opendir` obre un directori, `readdir` retorna una `struct dirent` per entrada (o `NULL` al final) i `closedir` el tanca. Les entrades arriben en l'ordre intern del sistema de fitxers —a ext4, l'ordre d'un índex de dispersió— i inclouen `.` i `..`. De `struct dirent` importen `d_name`, `d_ino` i `d_type`; aquest últim estalvia un `stat`, però pot ser `DT_UNKNOWN` en alguns sistemes de fitxers, i llavors cal `lstat`. En recórrer un arbre, mai no se segueixen els enllaços simbòlics a directoris: podrien formar cicles.

C tds/td2/myls.c

```

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue with the
        others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");

```

Sortida real

```

$ ./myls -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..

```

§7 Més enllà: registre de diari, còpia en escriptura i `fsync`

- **Registre de diari.** Crear un fitxer modifica diversos blocs (mapa de bits, inode, directori). Si el corrent falla a mig camí, el sistema de fitxers queda inconsistent. ext4 i XFS escriuen primer l'operació en un diari i després l'apliquen: en reiniciar n'hi ha prou de repetir o descartar el que estava pendent.
- **Còpia en escriptura.** Btrfs i ZFS mai no sobreescruien un bloc al seu lloc: n'escriuen un de nou i canvien el punter. Això dona instantànies gairebé gratuïtes i sumes de verificació de cada bloc.
- **Memòria cau de pàgines i `fsync`.** `write` torna quan les dades són a la memòria cau del nucli, no al disc. Un programa que necessita durabilitat (una base de dades, un editor que desa) crida `fsync`, i perquè el *nom* també sigui durable, `fsync` sobre el directori.
- **FUSE.** Un sistema de fitxers es pot implementar com un procés d'usuari (sshfs, rclone): el nucli li reenvia les crides.